

Sim-Engine Secret Sauce, Explained

What six physics engines actually do under the hood — read from their source code — and what that means for a two-robot combat-RL stack that has hit the limits of MJX. Companion to [notes/sim-engine-secret-sauce.md](#), which carries the full file-and-line citations.

This project trains four-legged fighting robots: two of them, in one arena, millions of times in parallel on an A100. That workload has produced five very specific complaints against the current MuJoCo-MJX-brax stack — and an itch to build a bespoke simulator that "compiles from a strictly typed language down to the simplest GPU operations." Before building anything, we read the source of every engine that matters. The short version: **the bespoke compiler already exists** (NVIDIA Warp), **the bespoke engine is already being built in the open** (mujoco_warp, Newton), and every one of the five complaints is either fixed there or fixable with an afternoon of model surgery. The long version follows, with diagrams.

The five complaints, and where each one ends up

Pain point	Root cause	Resolution
Contact cost scales with <i>possible</i> pairs, not actual ones	MJX freezes the full pair list into the compiled program (§3)	mujoco_warp compacts to actual pairs with atomics (§5)
<code>mjx.ray</code> silently ignores heightfields (lidar)	Its dispatch table simply has no HFIELD entry (§3)	Done in mujoco_warp, incl. mesh BVH (§5); port = contribution #2
Slider-crank leg needs <code>dt=0.002</code> or explodes	Soft <code><connect></code> = explicit stiff spring at a singular Jacobian (§9)	Quartic joint coupling — zero engine code (§9)
JIT compiles take minutes	Whole-pipeline XLA trace, every shape baked in (§3)	Warp: per-module compile, ~5 ms warm cache (§4)
brax wrapper bugs	brax sits on the JAX backend of MJX	Direct mjwarp / mjlab / Newton paths (§5–6)

1 · The map: who sits on what

Almost everything in this story is one family. MuJoCo's C engine defines the physics *semantics*; MJX re-expresses them for JAX; mujoco_warp re-implements them as explicit GPU kernels on Warp; Newton wraps mujoco_warp as its primary solver and adds seven more; Isaac Lab rides PhysX today and is adopting Newton. Genesis stands apart on its own compiler fork. Everything green-to-aqua below is Apache-2.0 open source.

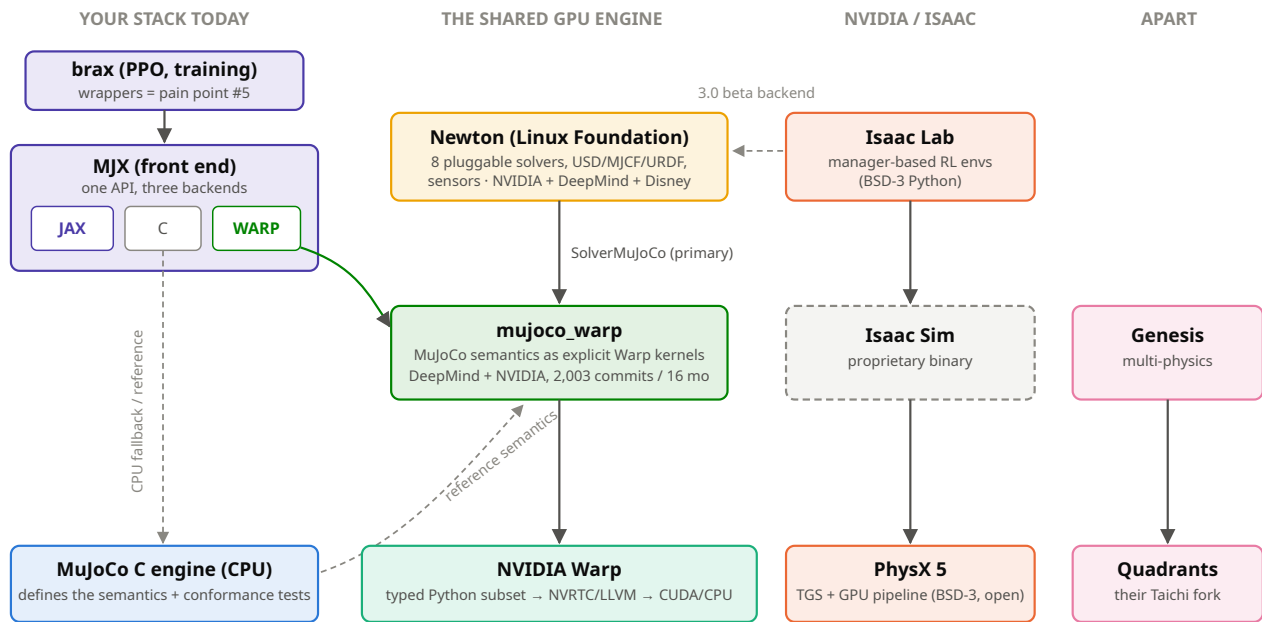


Fig. 1 — The 2026 GPU-simulation family tree. Solid arrows = "runs on"; dashed = reference or in-progress. The WARP chip inside MJX is the escape hatch: same API, mujoco_warp underneath.

Vocabulary in six boxes

solref / solimp (MuJoCo). Every constraint — contact or equality — is a virtual spring-damper whose *impedance* d varies with violation depth and is hard-clamped below 1. Consequence: finite stiffness everywhere, so the solve is a strictly convex optimization with bounded forces; the price is drift (constraints are never exact).

PGS vs Newton (solvers). PGS sweeps constraints one at a time (cheap iterations, slow convergence). MuJoCo's Newton solver descends the whole convex problem with an exact Hessian — few, expensive iterations, made cheap on CPU by sparse factorization plus rank-1 updates when the active set barely changes.

TGS (PhysX). Splits the timestep into sub-steps *inside* the solver: each iteration moves the bodies a little, re-measures constraint error, and integrates springs backward-Euler. Stiffness that would explode an explicit integrator becomes unconditionally stable.

XPBD (Newton). Solves constraints directly at the *position* level each substep with compliance (inverse stiffness) and accumulated multipliers. Also stiffness-unconditionally-stable; accuracy is bought with substep count.

Featherstone / reduced coordinates. Parameterize by joint angles so joints are exact by construction — but the mechanism must be a tree. Closed loops need a cut joint plus a constraint (all engines here), or coordinate elimination (none, automatically).

Dense vs sparse etc. MuJoCo-family solvers build a constraint Jacobian whose rows are constraint equations. Cost follows *allocated* rows — which is why MJX's static worst-case allocation hurts and mujoco_warp's runtime compaction doesn't.

2 · MuJoCo: why it's fast on a CPU, and why GPUs hate it

MuJoCo rebuilds its entire contact and constraint world from scratch every step — and is fast *because* of that, not despite it. The per-step pipeline:

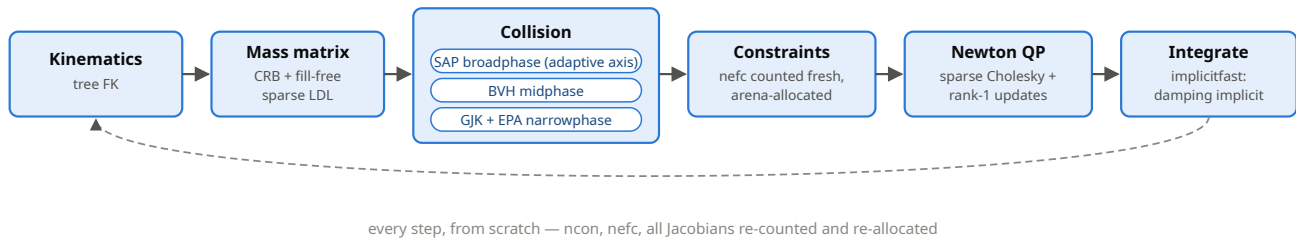


Fig. 2 — MuJoCo's step (`engine_forward.c mj_fwdPosition`). Nothing persists between steps except the state vector.

The contact model. Broadphase is a sweep-and-prune whose sweep axis *adapts to the scene* — it eigen-decomposes the covariance of all geom centers each step and sorts along the dominant axis. Narrowphase dispatches through a function table: analytic formulas for primitive pairs (sphere, capsule, box...) and an in-house GJK+EPA with polygon-clipped multi-contact for convex shapes. **The constraint model** is the famous soft one: $\text{force} \approx -K \cdot d(r) \cdot \text{violation} - B \cdot \text{velocity}$, where impedance $d(r) < 1$ always, so the regularizer $R > 0$ keeps the optimization strictly convex — *constraint forces are finite by construction, even at a singular mechanism Jacobian* (this matters in §9). The Newton solver then converges in a handful of iterations because the problem is convex and its Hessian exact; between iterations it never refactorizes, applying rank-1 Cholesky updates only for constraints whose active state flipped.

Why this resists GPU-ification. Four structural reasons, all visible in the code:

1. **Dynamic shapes everywhere** — contact and constraint counts are recomputed and arena-allocated per step; GPUs want static shapes.
2. **Branchy narrowphase** — a function-pointer table into wildly different algorithms with data-dependent iteration counts; hostile to SIMT warps.
3. **Tree-ordered sparse algebra** — the mass-matrix factorization walks parent chains with loop-carried dependencies.
4. **Stateful active-set logic** — the rank-1-update trick is inherently sequential.

Every GPU engine in this guide is one answer to the question: *which of these four do you give up, and what do you pay?*

3 · MJX: the quadratic two-robot bill, itemized

MJX maps MuJoCo onto JAX by accepting rule #1 above completely: **every shape is frozen at trace time**. Its collision driver enumerates all statically-possible geom pairs in Python while tracing — nested loops over bodies, filtered only by static rules (excludes, weld, parent-child, contype/conaffinity masks). There is no positional filtering: every surviving pair gets its narrowphase function vmapped over it *every step*, touching or not, and every pair gets contact slots and dense constraint-Jacobian rows allocated in the compiled program. Deactivation happens after the FLOPs are spent, by multiplying rows to zero.

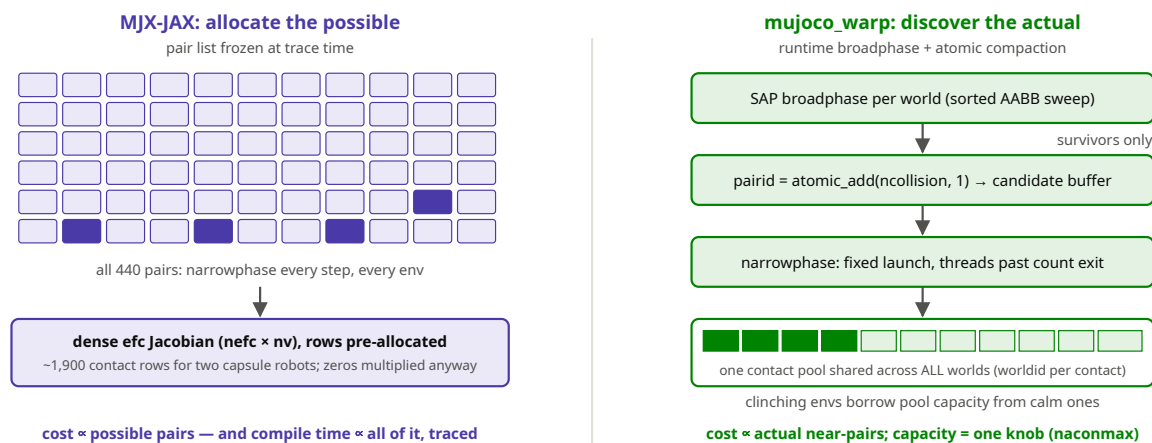


Fig. 3 — The core architectural difference. Left: MJX evaluates the shaded grid (every statically possible pair) each step; dark cells are the few that actually touch. Right: mujoco_warp's pipeline touches only what the broadphase finds, compacting with atomics into a shared pool.

The bill for this project. Two robots × ~20 collision geoms (torso box, four legs of hip/thigh/calf capsules + foot spheres, optional striker) live in different kinematic trees, so no static filter applies between them: 20×20 = **400 cross-robot pairs** plus ~40 floor pairs. All capsules → ~480 contact slots → at condim=3 pyramidal friction, four Jacobian rows each: **~1,900 dense constraint rows per env-step**, multiplied by 8,192 envs, whether or not the robots are anywhere near each other. The hand-tuned contype/conaffinity masks already in `gen_robot_mjcf.py` (the "F-SPEED" mode) are the only real lever, because they shrink the *trace-time* list itself.

Candidate pairs hitting narrowphase, per env-step

two 20-geom robots + floor · derived in note §2.2 (MJX) / §4 (mjwarp, typical mid-fight)

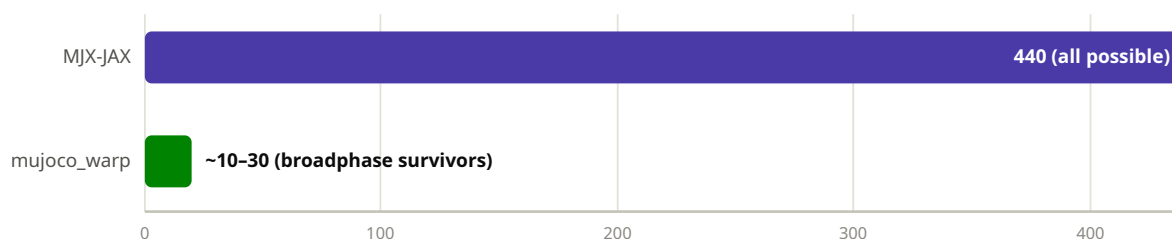


Fig. 4 — Same scene, same step, $\sim 20\times$ fewer narrowphase evaluations — and the gap widens with geom count (quadratic vs $\approx$ linear).

Two more MJX facts worth internalizing. First, `mjx.ray`'s dispatch table covers exactly six geom types — plane, sphere, capsule, ellipsoid, box, mesh (brute-force, no BVH). Heightfields aren't in the table and there is no error path: a lidar ray aimed at terrain *silently returns "no hit."* Second, the minutes-long JIT is structural, not incidental: the entire collision table, nine constraint builders, the Newton solver with linesearch, and the integrator trace into one XLA graph whose every shape is a constant, and convex colliders specialize per distinct mesh. Nothing inside MJX shortens it; only caching around it helps.

If staying on MJX-JAX for now: keep extending the contype masks; add the experimental `max_geom_pairs` / `max_contact_points` numerics (top-k sphere-distance culling); run Newton with `iterations=1`, `ls_iterations=4-6`, dense Jacobian — the docs recommend exactly this for RL. And know the escape hatch shipped in the same package: `mjx.put_model(m, impl='warp')` swaps the backend to `mujoco_warp` under the same API (no autodiff through physics; fine for PPO).

4 · Warp: the "strictly typed language → GPU" compiler already exists

The bespoke-simulator dream names a compiler: a strictly typed language lowered to plain GPU kernels. That is literally NVIDIA Warp, five years old, Apache-2.0, DCO-only contributions. A `@wp.kernel` function must annotate every argument (missing annotations are a hard compile error); only 24 whitelisted Python AST node types are accepted — no lambdas, comprehensions, exceptions, recursion, or dynamic data structures; variables cannot change type; in-kernel allocation doesn't exist. What survives those rules compiles to a C++/CUDA source string via literal templates, then through NVRTC (GPU) or an embedded Clang/LLVM (CPU), keyed by a SHA-256 content hash into an on-disk cache.

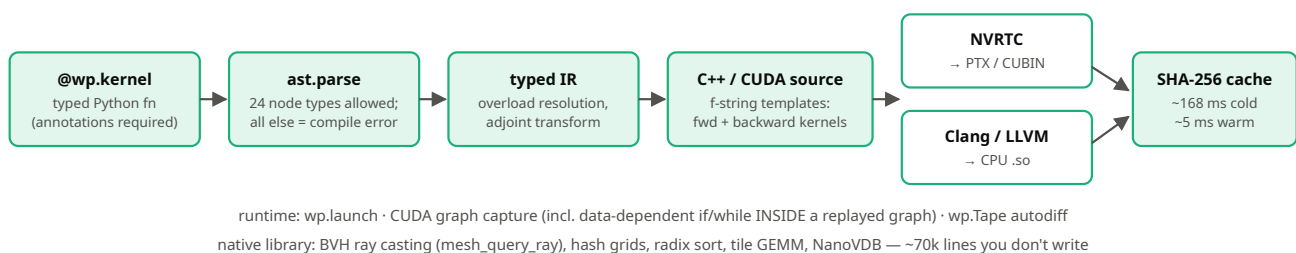


Fig. 5 — Warp's compile path. Per-module C++ translation units instead of one whole-program XLA graph: that single difference is "seconds vs minutes."

Scale check: ~7,000 commits, 100 contributors, roughly eight sustained core developers over 5.3 years — order of **20–30 person-years**, most of it in exactly the parts a from-scratch build would need first (adjoint codegen, CUDA graphs, BVH, kernel caching). Writing "a Warp" is not the project; writing *kernels* in Warp is.

5 • mujoco_warp: MuJoCo semantics rebuilt as explicit kernels

mujoco_warp (DeepMind + NVIDIA, 2,003 commits in 16 months, versioned in lockstep with MuJoCo) answers the four GPU-hostility problems of §2 one by one. Dynamic shapes → fixed-capacity pools filled by atomics, with per-world overflow flags instead of corruption. Branchy narrowphase → GJK/EPA rewritten as Warp device functions dispatched from a static table. Sparse tree algebra → dense tiled Cholesky (small robots) or a sparse path ($nv > 32$). Stateful solver logic → a CUDA-graph `capture_while` loop in which *each world exits the Newton solve the moment it converges*, decrementing a global counter that ends the loop when all are done. The whole step — collision, solve, integrate — is captured once as a CUDA graph and replayed; a batch of worlds costs one launch.

The two headline features for this project: **heightfield rays and rangefinder sensors are implemented** (BVH-accelerated mesh rays too — the GitHub issues for hfield ray are all closed), and **domain randomization without recompiles**: Model fields carry a broadcastable leading dimension, so per-world masses, frictions, and gains are an array shape, not a new program. Missing: differentiability (open issue #500), PGS/noslip, plugins.

Caveat before migrating the mesh leg: open issue #1270 — connect-constraint anchors don't account for joint reference positions. The slider-crank leg is a ready-made repro; test it first, and see §9 for why you may not need `<connect>` at all. Related: #1415, the blocked Cholesky lacks a pivot floor and can NaN near singular configurations — the dead center again.

6 · Newton: the bespoke engine, already being built in the open

Newton (Linux Foundation; initiated by NVIDIA, Google DeepMind, and Disney Research; v1.0 in March 2026, ~2,100 commits) is what "build a modern GPU sim on Warp" looks like when twenty funded engineers do it: one Model/State/Control data layer holding *both* maximal and generalized coordinates, with eight interchangeable solver plugins on top and USD/MJCF/URDF importers of unusual fidelity (tendons, heightfields, equality constraints — the latter converted to native loop joints for solvers that want them).

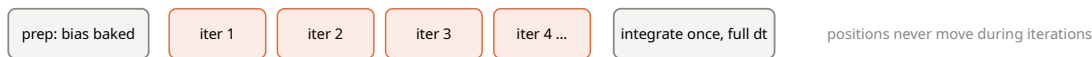
Solver	What it is	Relevance here
SolverMuJoCo	full bridge to mujoco_warp (builds an MjSpec, defaults Newton + implicitfast)	"primary backend" — the drop-in path
SolverXPBD	position-based, maximal coords	native loop joints, stiffness-stable
SolverVBD / AVBD	vertex block descent, ramped penalties	experimental
SolverFeatherstone	reduced-coordinate CRBA	silently ignores loop joints — never for this leg
SolverKamino	Disney's proximal-ADMM NCP solver; <i>built for kinematic loops + hard contacts</i>	Beta; the most principled dead-center answer in this survey
SemiImplicit / Style3D / ImplicitMPM	legacy rigid / cloth / granular	n/a

Sensors include a Warp-native tiled-camera raytracer and per-primitive analytic raycast functions — but **no turnkey lidar sensor** (all the pieces exist; composing them is contribution #5 in §11). Governance caveats: committers are overwhelmingly NVIDIA (DeepMind's share flows through mujoco_warp), contributions need a CLA, and several solvers are still flagged experimental. Isaac Lab 3.0 Beta ships Newton as an experimental backend with validated PhysX ↔ Newton policy transfer.

7 · PhysX 5: the Isaac secret sauce is temporal

PhysX wins stiff-constraint stability a completely different way: **everything is cached, substepped, and re-measured over time** rather than re-derived from scratch. Three mechanisms carry most of it:

PGS: iterate against a frozen error estimate



TGS: each iteration is a real sub-timestep



result: stiffness-1e4 joint drives stable at $dt = 1/60$ — the reason Isaac trains at large timesteps

Fig. 6 — Temporal Gauss-Seidel vs classic projected Gauss-Seidel. TGS re-integrates poses inside the solver loop and solves springs implicitly per substep.

- **Aggregates:** a whole robot is one broadphase entry with its own local mini-sweep for self-pairs — a 30-link self-colliding robot costs the global broadphase one box.
- **Persistent contact manifolds:** up to four cached contact points per pair, stored in body-local space, refreshed by re-projection each frame and regenerated only when drift exceeds thresholds — most frames do no full contact generation at all.
- **Dynamic GPU contact streams:** an incremental GPU sweep-and-prune reports only found/lost pairs; real contacts bump-allocate into capacity-limited device buffers that warn and drop on overflow. Memory follows expected load, not possible pairs — the same philosophy as mujoco_warp, five years earlier. Environment-ID filtering lives *inside* the broadphase kernel: cross-env pairs are rejected before they exist.

Isaac Lab adds config-driven MDP managers, cloning with env-ID isolation, PhysX contact-report sensors, tiled cameras — and a ray-caster that is just Warp's `mesh_query_ray` against a *single static mesh*: fine for terrain height-scans, useless for sensing the opponent robot. All of Isaac Lab's Python is BSD-3, but it imports the proprietary Isaac Sim binary to run; PhysX itself is BSD-3 with its GPU kernels now open in-tree.

8 • Genesis: an honest read

Under the hood, Genesis is a Taichi program whose team now maintains the compiler themselves: upstream Taichi stalled, so they forked it (June 2025) and ship it as "Quadrants." The rigid-body solver is an *openly acknowledged* GPU-batched MuJoCo reimplementation — solref/solimp tuples, CG/Newton solvers, EPA ported line-for-line (the source cites the MuJoCo C file it mirrors), MuJoCo-consistency tests in CI, and the same connect/weld/joint equality trio. It has a real LBVH lidar sensor and Isaac-Gym-derived terrain tools.

The December 2024 launch claimed "430,000× faster than realtime" and "10–80× faster than existing GPU sims." The number was real batched throughput for a contact-light, mostly-static scene (30k Franka envs, one substep, self-collision off); under corrected settings it dropped ~150×, and on contact-rich manipulation Genesis measured 3–10× *slower* than ManiSkill-class sims. The authors later published corrected open-script benchmarks and removed all speed claims from the README (May 2026). Today it is professionally maintained (a full-time team under the Genesis AI company; ~1,500 commits), Apache-2.0 with no CLA, and its genuine differentiator is multi-physics breadth (MPM, SPH, FEM, cloth), not rigid-body speed. Structural risk: a small team owning an entire compiler fork.

9 · The toggle problem: the dead center, engine by engine

Each leg of the mesh robot closes a slider-crank loop with a MuJoCo `<connect>` weld between the blade and the pushrod. At top-dead-center ($\varphi = 0$) the crank and conrod are collinear: the loop Jacobian goes singular, the constraint force direction flips as the piston reverses, and at $dt = 0.004$ the simulation measured 10 m of slide error and $|qacc| \approx 7 \times 10^9$. At $dt = 0.002$ it tracks the closed form to under a millimeter — hence the whole fleet paying double the steps.

Why it explodes even though MuJoCo's forces are "bounded." The per-step convex solve does guarantee a finite force (§2). But that force is applied *explicitly across the step* — the constraint enters the integrator as a plain right-hand-side force, outside the implicit treatment `implicitfast` reserves for damping. So the soft connect behaves as a stiff discrete-time oscillator; near TDC its effective inertia collapses (50–80 g linkage bodies, near-singular J) while the regularizer that is supposed to match — scaled from inverse weights precomputed *at the reference pose* — no longer fits the configuration. Halving dt restores sampling of the force-direction flip. (Code facts verified; the instability chain is analysis.)

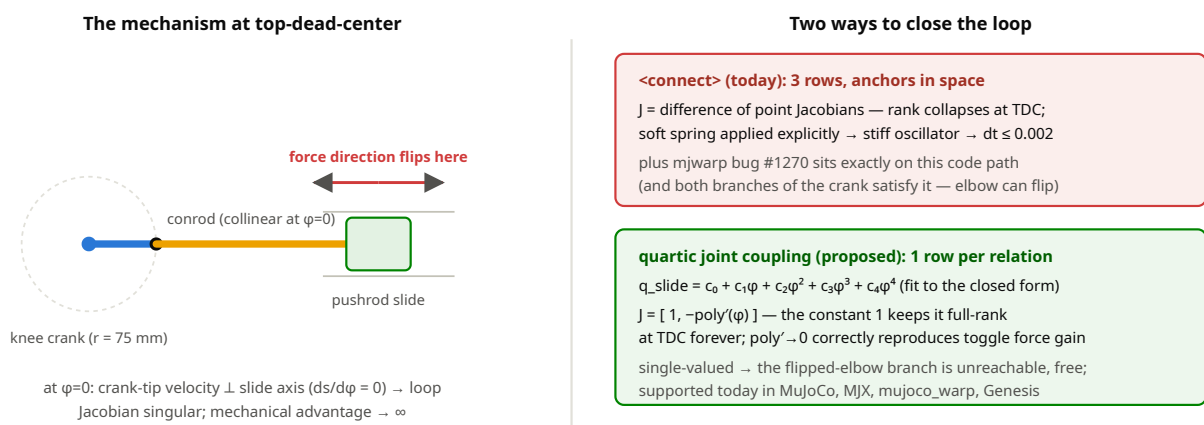


Fig. 7 — The dead-center problem and the model-level fix: replace the spatial weld with polynomial couplings between the joints themselves.

The afternoon experiment: fit the quartic against the closed-form `slider_crank_s( $\varphi$ )` already in `gen_mesh_robot_mjcf.py` (weight the TDC neighborhood), one coupling for the slide and one for the toe hinge, delete the `<connect>`, and retry $dt = 0.004$. Gates: fit residual < 0.5 mm over the working range; the loop-consistency contract test; the zero-g whip test; and the toggle-press force profile against the ~ 1.2 kN connect response. If the quartic can't hold tolerance, MuJoCo's tendon equality allows piecewise shaping.

How each engine treats a dead-center mechanism

Engine	Loop closure	Dead-center behavior
MuJoCo / MJX / mjwarp	soft equality (acceleration level)	forces finite, positions drift; explicit across steps → dt-limited near the singularity (the measured blowup)
PhysX TGS	maximal joint between articulation links	position-level + implicit springs per substep: no global dt cut, but the loop joint is documented as the largest error accumulator; mimic joints are linear-gear only
Newton XPBD	native loop joints, compliance + multipliers	stiffness-unconditionally stable; accuracy bought with substeps; approximate momentum
Newton Kamino	NCP via proximal ADMM, built for loops	the principled answer; Beta — watch it
Newton Featherstone	silently not enforced	trap — never for this leg
Genesis	MuJoCo equality trio	same as MuJoCo
<i>Coordinate elimination</i>	bake $\psi(\varphi)$, $s(\varphi)$ into kinematics	exact, singularity-free, no constraint at all — nobody does it automatically; the quartic coupling is its 95% stand-in, available today

10 · Licenses, plainly

Repo	License	To contribute
MuJoCo + MJX	Apache-2.0	Google CLA (one-time signature)
mujoco_warp	Apache-2.0	Google CLA; friendly tooling, expects external PRs
Warp	Apache-2.0	DCO only (<code>git commit -s</code>) — lowest friction
Newton	Apache-2.0	CLA (Linux Foundation governance)
PhysX	BSD-3	DCO; GPU kernels now open in-tree
Isaac Lab	BSD-3	DCO — but runtime needs proprietary Isaac Sim
Genesis	Apache-2.0	plain PRs, no CLA/DCO

For a project that prefers MIT: **for using these engines, Apache-2.0 and BSD-3 are practically identical to MIT** — permissive, no copyleft, commercial and closed use fine, and your own code stays MIT. The differences that exist are mostly in your favor: Apache-2.0 carries an explicit patent grant (protection you *want* from NVIDIA- and Google-owned code) plus a keep-the-LICENSE/NOTICE-files obligation; BSD-3 adds only a don't-use-our-name clause. You cannot relicense *their* files as MIT, but you can depend on them, vendor them, or ship them inside an MIT project with attribution.

11 · Verdict: contribute, don't rebuild

(a) What "bespoke" actually costs. The compiler half is solved and reusable (Warp, §4). The engine half, measured on a team *re-implementing known semantics with a conformance oracle*, is 10–15 person-years and counting — and still missing autodiff. A from-scratch engine without a reference implementation buys back none of that.

Estimated engineering effort to date (person-years)

from git history: commits, sustained contributors, duration · whiskers = range

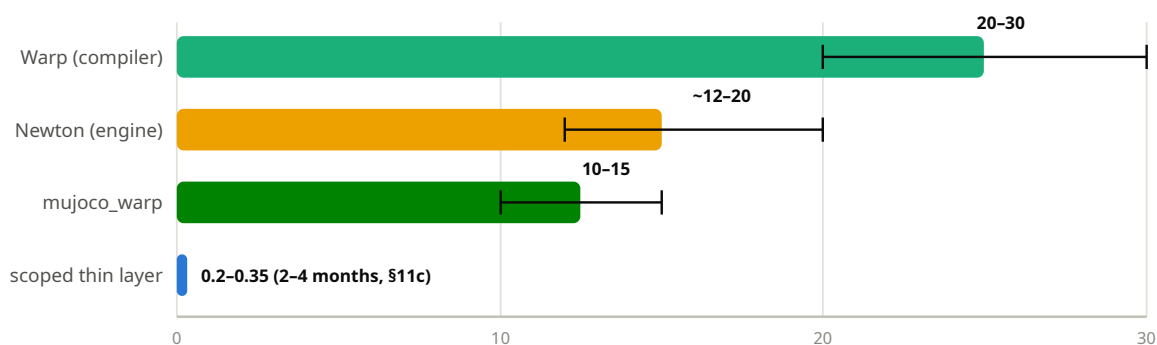


Fig. 8 — The asymmetry that decides the build-vs-contribute question. (*PhysX*, at 18+ years of development, doesn't fit on this axis.)

(b) The pragmatic ladder.

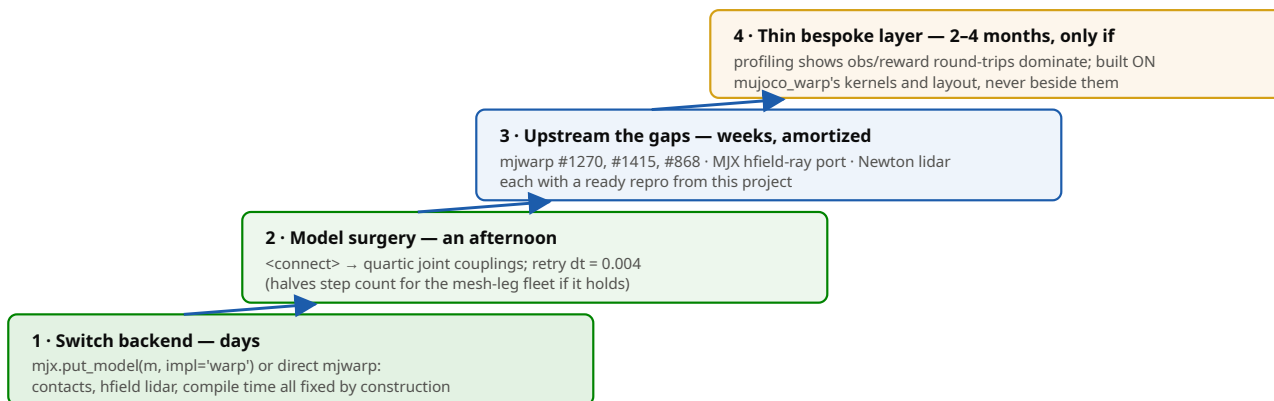


Fig. 9 — Each rung is independently useful; stop climbing when the training run is fast enough.

The five named contribution targets, with the files they touch:

1. **mjwarp #1270** (connect anchors vs joint reference positions) — `mujoco_warp/_src/constraint.py`, regression test against the leg's closed form. De-risks the GPU migration of the mesh robot.
2. **MJX heightfield ray** — port `mujoco_warp/_src/ray.py` `ray_hfield` into `mjax/_src/ray.py`'s dispatch table (and add a warning for unsupported types). Unblocks lidar-over-terrain in the current brax stack.

3. **mjwarp #1415** (Cholesky pivot floor) — `mujoco_warp/_src/block_cholesky.py`. Solver stability exactly at near-singular configurations like TDC.
4. **mjwarp #868** (incremental Hessian) — port the C engine's rank-1 update trick (`engine_solver.c HessianIncremental`) to `_src/solver.py`. Big win for contact-rich two-robot worlds.
5. **Newton lidar sensor** — compose `newton/_src/geometry/raycast.py` into a `SensorLidar` next to the tiled camera. Fills a documented gap.

(c) If the thin layer is ever built, its genuinely bespoke 10% is four small things: an exact loop-coordinate joint ($\psi(\phi)$, $s(\phi)$ baked into kinematics — ~200 lines, the one thing no engine offers), a hand-curated static pair table (~60–120 pairs with analytic capsule kernels — at this scene size, curated-static beats any broadphase), a 144-ray lidar kernel (~200 lines), and obs/reward kernels fused into the captured CUDA step graph (the actual payoff: no device ↔ host round-trip per control step). Everything else — compiler, contacts, solver, integrator, rays, autodiff — is reuse. Milestones with kill-criteria are in the companion note, §10c.

One line to remember: the bespoke compiler exists, the bespoke engine is being built in the open by twenty funded engineers, and this project's genuinely novel needs total a few hundred lines on top of that stack — not a simulator instead of it.

Companion note with full file-and-line citations: `notes/sim-engine-secret-sauce.md` (commit-pinned: mujoco 14c0b0c9, mujoco_warp 3.10.0.1, newton 1.4.0.dev0, Genesis v1.2.1). Method: seven parallel code-reading threads over fresh clones of mujoco, mujoco_warp, warp, newton, PhysX, IsaacLab, Genesis; claims verified with grep against the checkouts; web checks for governance, roadmaps, and the Genesis benchmark controversy. Charts use estimated ranges where noted; all hard numbers trace to the companion note.